ИТОГОВАЯ АТТЕСТАЦИОННАЯ РАБОТА

Программа обучения
«Python-разработчик с использованием инструментов ИИ»

# Генератор случайных паролей с оценкой сложности

*Кейс 8*

*PasswordGenerator 1.0.0*

ФИО слушателя: _____

Группа: ПР-5-Р

Период обучения: 05.10.2026–09.11.2026

Дата аттестации: _____

Преподаватель: _____

Исходный код
https://github.com/KirillakaCarl/PasswordGenerator

2026

## Описание кейса и выбор технологий

Разработано локальное графическое приложение для генерации паролей, оценки их сложности и сохранения в зашифрованном файле. Работа выполнена по кейсу 8. Дополнительно реализованы поиск по названию, дата сохранения и сортировка записей.

Основные задачи: дать пользователю выбор длины и групп символов; объяснить оценку сложности; обеспечить зашифрованное сохранение; проверить алгоритмы тестами и показать фактическое применение ИИ при разработке.

| **Инструмент** | **Причина выбора** |
|---|---|
| Python и Tkinter | Изучены на курсе; позволяют создать локальный графический интерфейс. |
| secrets | Источник случайности для генерации секретных значений. |
| cryptography | Готовые реализации Argon2id и Fernet вместо собственного шифрования. |
| unittest и Git | Проверки правил, история изменений, ветки и самостоятельные Pull Request. |
| PyInstaller | Windows exe со встроенным Python и зависимостями. |

## Архитектура приложения

main.py запускает окно. gui.py и storage_ui.py обрабатывают действия пользователя. generator.py, assessment.py и entry_view.py содержат независимые алгоритмы. encryption.py получает ключ и проверяет токен, storage.py проверяет схему файла и сохраняет записи.

Разделение позволяет проверять правила без графического интерфейса. Тяжёлая работа с файлом выполняется в фоновом потоке. Результат передаётся через очередь, а виджеты обновляются в основном потоке Tkinter.

## Генерация и пользовательский интерфейс

Пользователь задаёт длину от 4 до 128 и включает нужные группы: строчные латинские буквы, прописные буквы, цифры и специальные символы. После проверки ввода программа объединяет алфавиты и выбирает символы через secrets.choice.

Кандидат принимается только при наличии хотя бы одного символа каждой выбранной группы. Иначе создаётся новый кандидат. Группы не закрепляются за определёнными позициями. Повторное получение одинакового пароля математически возможно; уникальность не обещается.

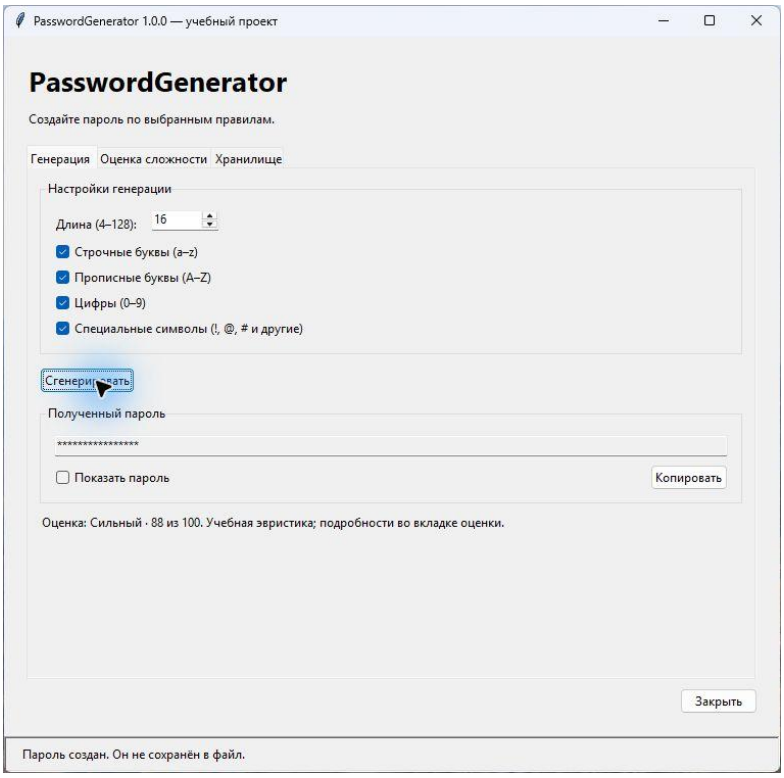



**Рис. 1. Генерация пароля длиной 16 по четырём группам символов.**

Результат изначально скрыт, доступен показ и явное копирование. Генерация не сохраняет пароль автоматически. Ошибки длины или пустого набора групп выводятся понятным сообщением.

## Оценка сложности и применение ИИ

Базовая шкала: min(60, 3 × длина) + 10 × число групп. Категории: 0–39 - слабый, 40–74 - средний, 75–100 - сильный. Короткая длина дополнительно ограничивает категорию. Затем применяются ограничения за известные слова, простые подстановки, повторы и последовательности.

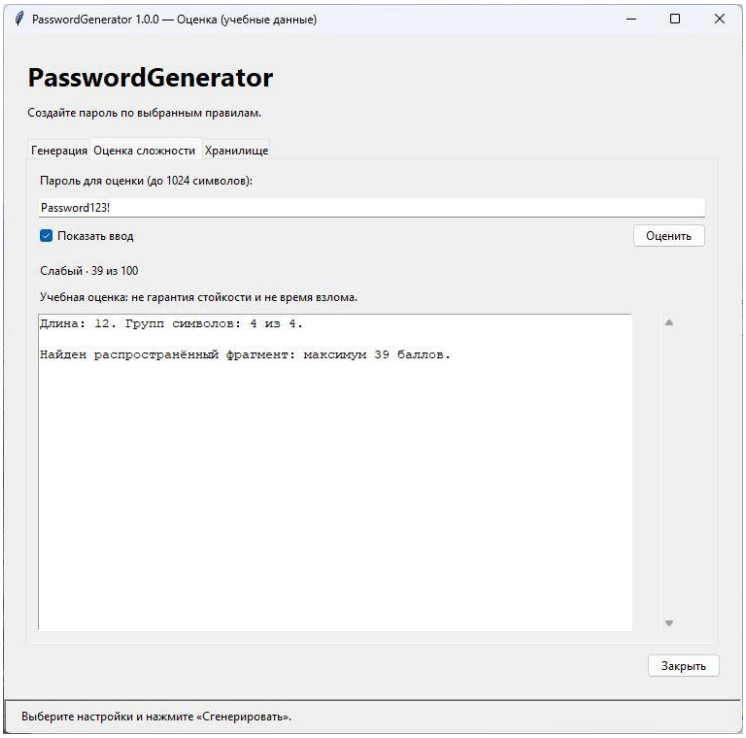



**Рис. 2. Учебный шаблон Password123! получает 39 баллов с пояснением.**

Codex помог выявить завышенную оценку шаблонов и предложить улучшения. На семи заранее описанных примерах совпадения с ожидаемыми категориями выросли с 4/7 до 7/7, на шести отложенных - с 2/6 до 6/6. Наборы подготовлены тем же ИИ: это проверка учебного контракта, а не независимое исследование безопасности.

В приложении нет вызовов ИИ. Реальные пароли не отправляются внешним сервисам. Формула, примеры и ограничения описаны в ASSESSMENT_RULES.md и AI_USAGE.md.

## Зашифрованное хранилище

Мастер-пароль длиной 12–1024 символа преобразуется в ключ через Argon2id: 64 МиБ памяти, 3 прохода, 4 линии. Для нового файла создаётся соль - 16 случайных байтов. Соль хранится открыто и вместе с тем же мастер-паролем позволяет снова получить тот же ключ. Мастер-пароль и ключ не записываются на диск.

Fernet шифрует названия и пароли вместе и проверяет подлинность при открытии. Перед вычислением ключа проверяются размер, версия и фиксированные параметры. Зашифрованные данные сначала пишутся во временный файл, затем публикуются без перезаписи существующего файла или атомарно заменяют прежнюю версию.

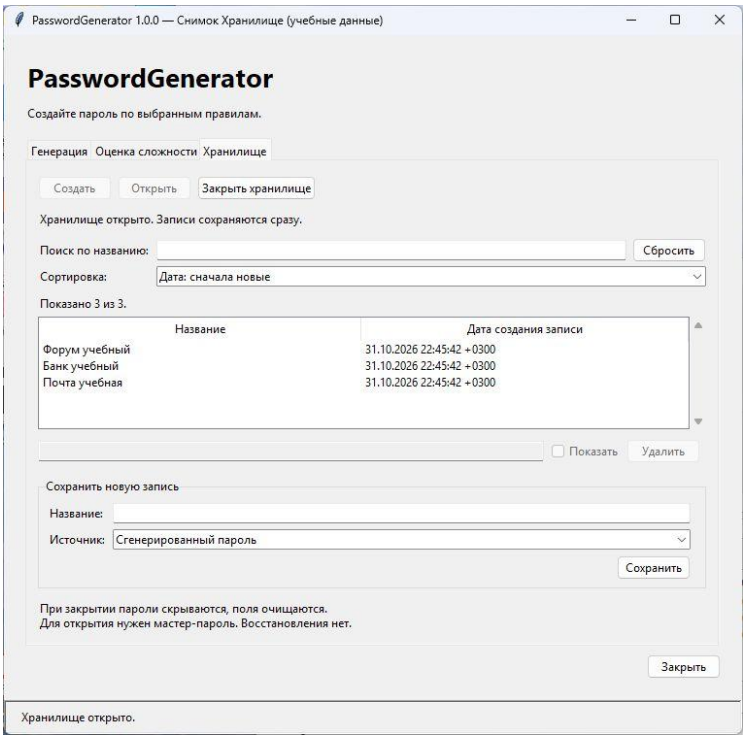



**Рис. 3. Хранилище с искусственными записями и датами сохранения.**

Поиск работает по части названия без регистра, сортировка - по названию и дате в обе стороны. Дата означает сохранение записи. При закрытии сеанса поля очищаются; для повторного доступа нужен мастер-пароль.

## Проверка и результаты тестирования

Пройдено 47 автоматических тестов. Используются только искусственные строки и временные файлы. Проверяются правила и ошибки, а не только успешные сценарии.

| Область | Тестов | Основные проверки |
|---|---|---|
| Генерация | 6 | Все 15 сочетаний групп; длины 4, 16, 128; неверные настройки. |
| Оценка | 10 | Шаблоны, границы категорий, Unicode, пробелы, пустой ввод. |
| Ключ и хранилище | 17 | Соль, неверный ключ, повреждение, схема, отказ записи, блокировка. |
| Мастер-пароль | 5 | Короткий ввод, предупреждение, подтверждение, повтор и отмена. |
| Поиск и сортировка | 9 | Регистр, отсутствие совпадений, пароли вне поиска, даты и порядок. |

Пользователь вручную проверил создание и закрытие хранилища, сохранение группы записей, прокрутку, неверный и короткий мастер-пароль, удаление, повторное открытие после перезапуска, поиск на русском и английском, цифры и специальные символы, сортировку и сброс фильтра.

## Самостоятельная сборка

PyInstaller 6.22.3 подготовил exe Windows x64 версии 1.0.0. Готовый exe проверен из отдельной папки с Python, исключённым из PATH. Встроенная проверка подтвердила Tkinter, генерацию, оценку, Argon2id, Fernet, сохранение и открытие, поиск, сортировку и удаление. Подготовлены ZIP, краткая инструкция и SHA-256. Такая проверка не доказывает совместимость со всеми компьютерами Windows.

История разработки сохранена в GitHub: функции выполнялись в отдельных ветках и объединялись через Pull Request. Учебные комментарии объясняют алгоритмы и причины решений.

## Заключение и инструкция запуска

Функции кейса 8 реализованы: настраиваемая генерация, объяснимая оценка, зашифрованное сохранение, интерфейс, тесты, документация и применение ИИ в разработке. Дополнительные поиск, даты и сортировка упрощают работу со списком записей.

Для запуска готовой версии распакуйте переносной ZIP и откройте exe. Python не требуется. Для исходного кода создайте виртуальное окружение, установите requirements.txt и выполните main.py. Полные команды находятся в README. Подробные действия пользователя приведены в отдельной иллюстрированной инструкции.

Ограничения: оценка является эвристикой без проверки утечек и полного словаря. Нет восстановления и смены мастер-пароля, совместной записи в файл и облачной синхронизации. Размер хранилища ограничен 5 МиБ. Полное обнуление строк Python в памяти не гарантируется. Буфер обмена не очищается автоматически.

В дальнейшем можно добавить смену мастер-пароля и расширить набор правил оценки. Эти функции не заявляются выполненными в версии 1.0.0.

## Использованные источники


1. Кейсы на выбор. Кейс 8 и общие требования к итоговой работе.

2. Памятка по итоговой аттестации. Структура отчёта и требования к видеозащите.

3. Python: secrets и tkinter. https://docs.python.org/3/library/secrets.html и https://docs.python.org/3/library/tkinter.html

4. cryptography: Fernet и Argon2id. https://cryptography.io/en/stable/fernet/ и https://cryptography.io/en/stable/hazmat/primitives/key-derivation-functions/

5. PyInstaller. https://pyinstaller.org/en/stable/operating-mode.html

6. Репозиторий проекта. https://github.com/KirillakaCarl/PasswordGenerator